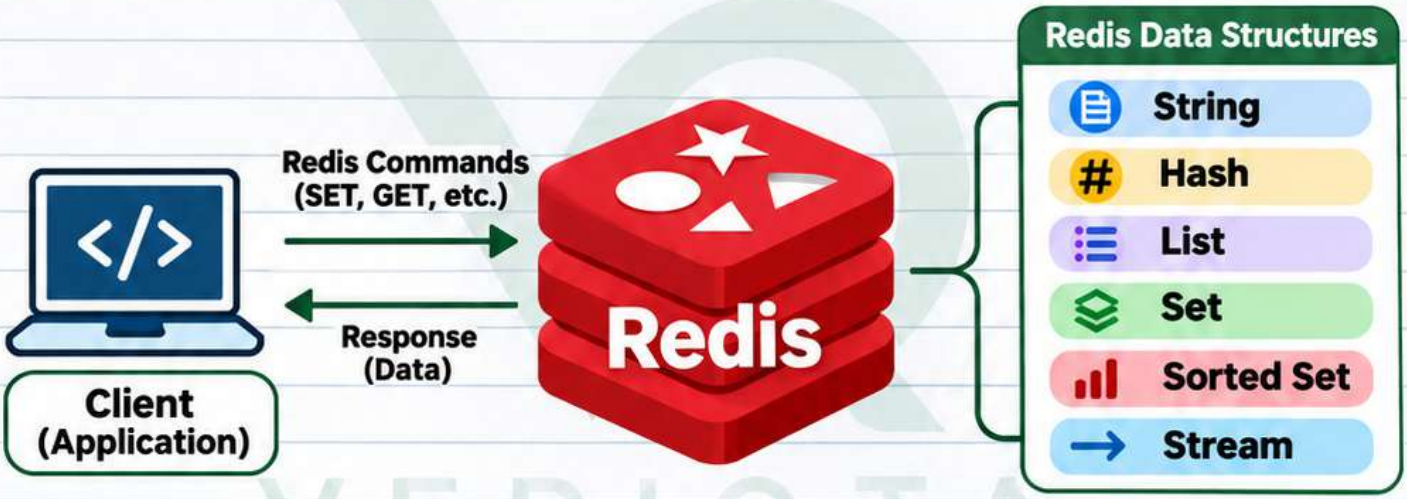
VERIQTA

Redis Under the Hood

A Practical Guide for Junior Engineers



VERIQTA

Fast In-Memory | Caching | Persistence (RDB / AOF) | Replication & High Availability | Scaling (Cluster) | Security (ACL, TLS)

What You Will Learn

- How Redis works internally
- Key features and use cases in real systems
- How to operate, secure and scale Redis
- How to troubleshoot common problems

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

What Is Redis and Why Does It Exist?

What Is Redis?

Redis is a fast, **in-memory data store** that keeps data in **RAM** for quick read and write access.



Why Does It Exist?

To give applications a faster way to store and access frequently used data than traditional databases.

The Problem It Solves

- Slow data access from disk
- High load on the main database
- Need for real-time data (e.g. sessions, cart, counters)
- Handling many requests at once

How It Fits in an Application



Common Use Cases

- Session storage (e.g. user login)
- Caching (e.g. product data)
- Real-time counters (e.g. page views)
- Rate limiting (e.g. API requests)
- Queues and real-time data (e.g. notifications)

Junior Engineer Insight

Redis is not a replacement for your main database. It works alongside it to make your application faster, more responsive and more scalable.

Try This

Start Redis locally with Docker and run a few basic commands using **redis-cli.**

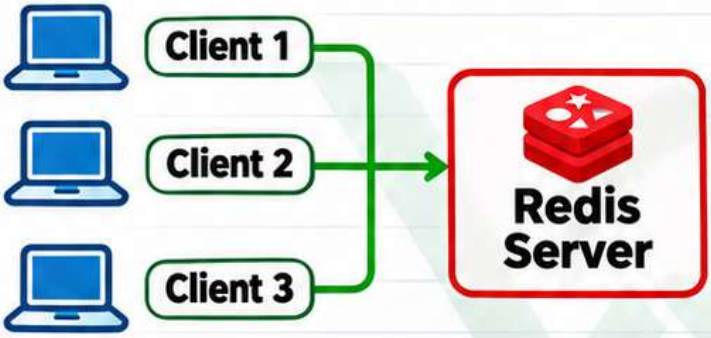
How Redis Works Under the Hood

The Big Picture

Redis is a high-performance, **in-memory** data store that processes commands quickly, stores data in **RAM** and handles **many client connections** at the same time.



Redis Architecture (Simple View)



Inside Redis

- **Event Loop** – handles many clients in a single thread.
- **In-Memory Storage** – keeps data in RAM for fast access.
- **Command Processor** – parses and executes Redis commands (e.g. SET, GET, DEL).
- **Persistence** – can save data to disk (RDB or AOF).

How a Redis Command Works



1. Client sends a command (e.g. SET)
2. Event loop receives and queues the command
3. Redis executes the command in memory
4. Result is sent back to the client

Handling Multiple Clients

- Uses a single-threaded event loop.
- Handles many connections concurrently using non-blocking I/O.
- This keeps Redis fast and predictable.

Key Takeaways

- Data lives in memory (**RAM**) for speed.
- An event loop processes commands efficiently.
- Redis can serve many clients at the same time with high performance.

Junior Engineer Insight

Redis is fast because it keeps data in memory and uses an **event-driven model**, instead of creating a new thread for every client.

Redis vs Traditional Databases

The Main Idea

Redis and traditional databases such as PostgreSQL or MySQL store data, but they are designed for different needs and workloads.

Redis | PostgreSQL | MySQL

Feature	Redis	PostgreSQL / MySQL
Data Storage	In memory (RAM)	On disk (persistent)
Speed	Very fast (microseconds)	Slower (milliseconds)
Data Structures	Multiple types (strings, hashes, lists, sets, sorted sets, streams)	Relational tables (rows and columns)
Persistence	Optional (RDB or AOF)	Built in and durable (ACID)
Typical Use	Caching, sessions, real-time data, counters, rate limiting	Long-term data, complex queries, reporting and transactions

When to Use Which?

Use Redis when you need:

- Very fast data access.
- Caching and session storage.
- Real-time features (e.g. leaderboards).

Use PostgreSQL / MySQL when you need:

- Durable, long-term storage.
- Structured relational data.
- Complex queries and transactions.

Key Takeaway

Redis is a high-speed, in-memory data store for performance and real-time workloads, while PostgreSQL or MySQL are persistent relational databases for structured, long-term data.

Redis Data Structures Explained

The Big Picture

Redis supports multiple data structures that let you store and work with different types of data efficientlv.



- Fast
- Flexible
- In-Memory
- Multiple Use Cases

String

- Stores a single value (text, number, or binary).

```
SET user:1 "Ann"
GET user:1
```

Example: Store a user name or a simple counter.

Hash

- Stores a collection of field-value pairs (like a small object).

```
HSET user:1 name Ann
HSET user:1 role devops
HGET user:1 role
```

Example: Store user details (name, role, email).

List

- Ordered collection of values (you can add or remove from both ends).

```
LPUSH tasks "build"
RPUSH tasks "deploy"
LRANGE tasks 0 -1
```

Example: Build a task queue or process a list of jobs.

Set

- Collection of unique values (no duplicates).

```
SADD users "ann" "john"
SMEMBERS users
```

Example: Keep a unique list of online users.

Sorted Set

- Like a set, but each member has a score (for ranking and ordering).

```
ZADD leaderboard 100 ann
ZADD leaderboard 80 john
ZREVRANGE leaderboard 0 -1
```

Example: Build a leaderboard (e.g. top scores).

Stream

- A durable, append-only log for real-time data (keeps a history of events).

```
XADD orders * id 101 item "laptop"
XRANGE orders - +
```

Example: Process real-time events (e.g. orders or logs).

When to Use Them

- String: simple values and counters.
- Hash: structured data (e.g. user profiles).
- List: queues and ordered tasks.
- Set: unique items (e.g. active users).
- Sorted Set: ranking/leaderboards.
- Stream: real-time event processing.

Key Takeaway

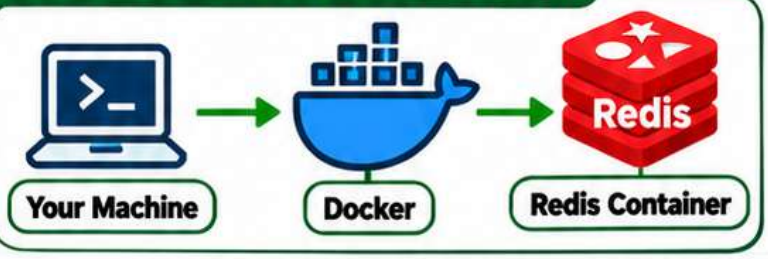
Redis gives you multiple data structures, so you can choose the right one for your application's data and performance needs.

Installing and Connecting to Redis

Overview

Run Redis locally with **Docker**, connect using **redis-cli**, test connectivity and understand basic configuration.

Architecture (Local Setup)



Step by Step

1. Pull the Redis image:

```
docker pull redis:7
```

2. Run Redis in a container:

```
docker run -d --name redis \
-p 6379:6379 redis:7
```

3. Check the container is running:

```
docker ps
```

4. Connect to Redis with redis-cli:

```
redis-cli
```

Test the Connection

```
> redis-cli
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> SET app veriqta
OK
127.0.0.1:6379> GET app
"veriqta"
```

Basic Configuration

- Redis uses the **redis.conf** file.
- Common settings:
  - **port 6379:** the listen port.
  - **bind 127.0.0.1:** the allowed IP.
  - **requirepass <password>:** adds a password for access.

Key Takeaways

- Docker makes it easy to run Redis locally.
- Use **redis-cli** to connect and run commands.
- **PING** checks if Redis is working.
- Redis can be configured with a **redis.conf** file.

Junior Engineer Tip

Use a password and restrict access **(bind)** when Redis is not just for local learning, especially in shared or cloud environments.

What Happens When You Run a Redis Command?

The Big Picture

A Redis command follows a simple path: your client sends a command over the network, Redis receives and executes it, accesses data in memory and returns a response.



- Send command
- Process command
- Access memory
- Return response

The Command Flow (End to End)



1. Client (redis-cli or app)
2. Network (TCP connection)
3. Command Processing (parse and execute)
4. Memory Access (read/write data in RAM)
5. Response (result sent back to the client)

Step by Step Explanation

1. **Client Sends Command** – you run a command (e.g. SET user:1 "Ann").
2. **Network Transport** – the command is sent to Redis over a TCP connection (e.g. port 6379).
3. **Redis Processes the Command** – Redis parses the command and checks if it is valid (e.g. SET, GET, DEL).
4. **Memory Access** – Redis reads or writes the data in RAM (in-memory data store).
5. **Response to Client** – Redis returns a result, such as OK, the stored value or an error.

Example in Action

```
> SET user:1 "Ann"
OK
> GET user:1
"Ann"
```

Key Takeaways

- Redis is event-driven and handles commands quickly.
- All data operations happen in memory (RAM).
- The client gets a response for every command.
- The whole process is fast, often in milliseconds.

How Redis Stores Data in Memory

The Big Picture

Redis keeps your data in RAM using efficient internal formats (encodings) to make it fast, compact and easy to access.

Redis

- Key–value store
- In-memory (RAM)
- Multiple data types
- Optimized encodings

How It Stores Data



Client
(SET user:1 "Ann")

→



In-Memory Data Store
(RAM)

→

Memory (Key–Value)

Key	Value
user:1	Ann

Internal Encodings

Redis uses different memory encodings based on the data type and size to save memory and improve performance.

Data Type	Example	Typical Encoding
String	"Ann"	int, embstr, raw
List	[a, b, c]	quicklist
Hash	{name, Ann}	listpack / hashtable
Set	{a, b, c}	intset / hashtable
Sorted Set	{score, member}	skiplist + hashtable

Memory Overhead

Redis uses extra memory beyond your data for internal management.

- Your Data (e.g. "Ann")
- Metadata (type, length, flags)
- Data Structure Overhead (e.g. hash table, pointers)

} **Total Memory Usage (> Data Size)**

Why Memory Usage Can Be Larger

- Redis adds metadata for each key and value.
- Some data types use extra structures (e.g. hash tables, skip lists).
- Small values may still take more space due to encoding and alignment.
- The more keys you store, the higher the overhead.

Key Takeaway

Redis stores data in RAM using optimized encodings, but the total memory usage is usually larger than the raw size of your data due to metadata and internal data structures.

Key Expiration, TTL and Eviction

The Big Picture

Redis can automatically remove keys after a time (TTL) and reclaim memory when it reaches a configured limit (eviction).



- Set a time to live (TTL)
- Keys expire automatically
- Redis reclaims memory
- Evicts keys when memory limit is reached

How Key Expiration (TTL) Works

```
SET session:123
"abc123"
EX 300
```

→ 

Key stored with TTL (300 seconds)

→ 

TTL counts down (300 → 0)

→ 

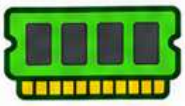
Key expires and is removed automatically

Key TTL Commands

Command	What It Does
EXPIRE key seconds	Sets a TTL in seconds.
PEXPIRE key ms	Sets a TTL in milliseconds.
TTL key	Shows remaining time (in seconds).
PTTL key	Shows remaining time (in milliseconds).
PERSIST key	Removes the TTL (makes the key permanent).

Eviction: When Memory Is Full

If Redis reaches its configured memory limit (maxmemory), it evicts (removes) keys based on the chosen eviction policy.



Memory Limit (e.g. 1GB)

→ 

Eviction Policy (e.g. LRU)

→ 

Remove keys to free memory

Common Eviction Policies

Policy	What It Does
noeviction	Rejects new writes (default).
allkeys-lru	Removes the least recently used key (from all keys).
allkeys-random	Removes a random key (from all keys).
volatile-lru	Removes the least recently used key (only keys with a TTL).
volatile-ttl	Removes the key with the shortest remaining TTL.

Key Takeaways

- TTL controls how long a key lives.
- Expired keys are removed automatically.
- Eviction frees memory when the memory limit is reached.
- Choose an eviction policy that fits your workload.

How Redis Caching Works

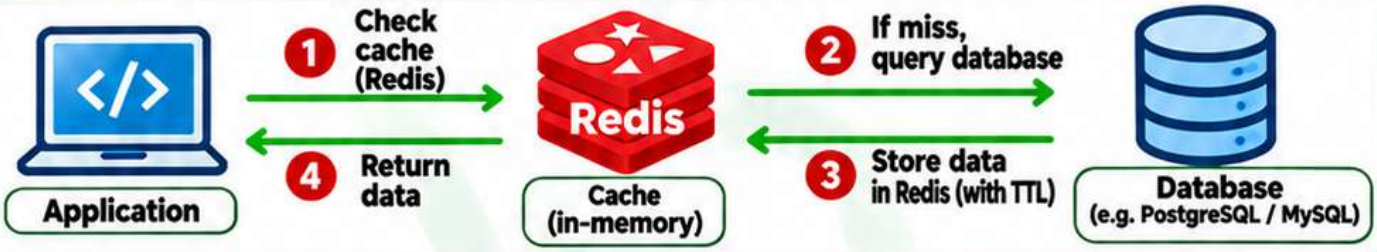
The Big Picture

Redis caching keeps frequently used data in memory, so your application can respond much faster without always querying the database.



- Stores frequently used data
- Reduces database load
- Speeds up response time
- Improves application scalability

Cache-Aside Pattern (How It Works)



Cache Hit vs Cache Miss

Cache Hit

- Data is found in Redis.
- Returned immediately (from memory).
- Very fast (milliseconds).

Cache Miss

- Data is not in Redis.
- Query the database.
- Store the result in Redis (with a TTL).
- Then return the data.

Step by Step: Cache-Aside Flow

1. The application requests data.
2. Check Redis for the data.
3. If found (hit), return it immediately.
4. If not found (miss), query the database.
5. Store the result in Redis with a TTL.
6. Return the data to the application.

Why Caching Improves Performance



Key Takeaways

- Caching keeps hot data in memory.
- Cache hits are fast and avoid database calls.
- Cache misses fetch data from the database and then store it in Redis.
- A TTL keeps the cache fresh and prevents stale data.
- The result is lower latency, higher throughput and a more scalable application.

Cache Invalidation and Stale Data

The Big Picture

Cached data can become outdated (stale) when the original data changes. Cache invalidation is the process of updating or removing cached data to keep it correct.



- Why data becomes stale
- How to invalidate cache
- Common strategies
- How to avoid consistency issues

What Is Stale Data?



Stale data is cached information that is no longer in sync with the latest data in the database.

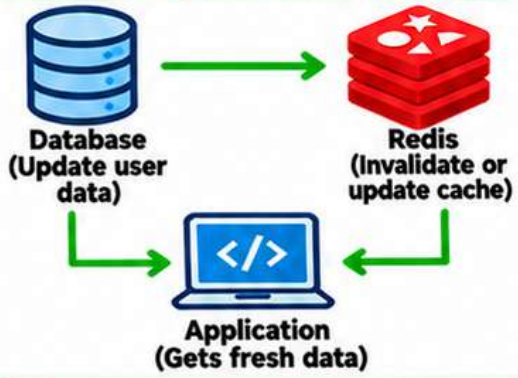
Why Cached Data Becomes Stale

- Data in the database is updated or deleted.
- The cache is not updated or cleared.
- A TTL is too long for frequently changing data.
- Multiple applications update the same data.

Common Cache Invalidation Strategies

Strategy	How It Works
TTL (Time to Live)	Let the cache expire automatically after a set time.
Delete on Update	Remove the cache key when the data changes (e.g. DEL key).
Update the Cache	Write the new value to the cache after updating the database.
Cache-Aside	On a miss, read from the database and refresh the cache.

Example: Update and Invalidate



Common Mistakes to Avoid

- Not invalidating the cache after data changes.
- Using a TTL that is too long for dynamic data.
- Invalidating the wrong keys.
- Ignoring cache updates in multi-service applications.

Key Takeaways

- Stale data happens when the cache and database are out of sync.
- Invalidate, update or expire cached data to keep it correct.
- Choose the right strategy based on how often your data changes.
- Test your cache logic, especially in distributed or multi-service systems.

Redis Persistence: RDB vs AOF

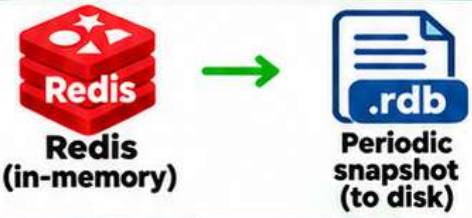
The Big Picture

Redis keeps data in memory for speed, but persistence saves your data to disk so it can be recovered after a restart or a failure.



- RDB: point-in-time snapshots
- AOF: log of every write
- Used for restart recovery
- Trade-off between speed, file size and data loss

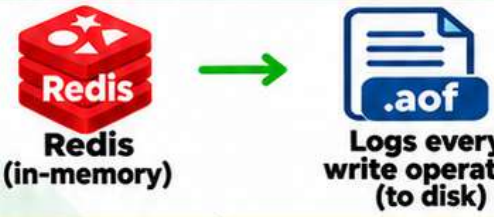
RDB (Redis Database) – Snapshots



Redis (in-memory) → Periodic snapshot (to disk)

- Saves a complete snapshot of the data.
- Creates an RDB file (e.g. dump.rdb).
- Fast to restore during a restart.
- May lose recent writes (data since the last snapshot).

AOF (Append Only File) – Write Log



Redis (in-memory) → Logs every write operation (to disk)

- Records every write command (e.g. SET, DEL).
- Stored in an AOF file (e.g. appendonly.aof).
- More durable and less data loss.
- Larger file size and slightly more overhead during writing.

RDB vs AOF: Quick Comparison

Aspect	RDB (Snapshot)	AOF (Write Log)
What it is	Point-in-time snapshot of the data	Log of every write command
File	dump.rdb	appendonly.aof
Data loss	Possible (since last snapshot)	Minimal (last few milliseconds to seconds)
Recovery speed	Faster	Slower (replay all commands)
File size	Smaller	Larger
Performance impact	Lower	Slightly higher
Best for	Backups, fast recovery, read-heavy workloads	Higher durability, data-sensitive workloads

Restart Recovery: How It Works



Trade-Offs and Data Loss



- RDB: better performance and smaller files, but you can lose recent data.
- AOF: higher durability and less data loss, but larger files and more I/O overhead.
- You can enable both (RDB + AOF) for a balance of speed and durability.

Key Takeaways

- Redis persistence keeps data after a restart.
- RDB creates snapshots; AOF records every write.
- RDB is faster and smaller; AOF is safer and more complete.
- There is a trade-off between performance, file size and data loss.
- Use the right option based on your workload and recovery needs.

Redis for Sessions, Counters and Rate Limiting

The Big Picture

Redis is a fast, in-memory data store that is commonly used to manage user sessions, keep atomic counters and enforce request limits (rate limiting).

- Store and expire user sessions
- Maintain atomic counters
- Limit the number of requests
- Handle high concurrency reliably

Redis for Sessions

Keep user login sessions (e.g. user ID, role, expiry)

```
SET session:abc123 "user:42" EX 3600
```

- Stores session data in memory.
- Uses TTL to expire sessions (e.g. 3600 seconds).
- Works well for distributed applications (e.g. multiple app servers).

Redis for Counters

000123

Track counts such as page views, likes or actions (e.g. login attempts).

```
INCR page:views:1001
```

- Increments a value atomically.
- Thread safe and works under high concurrency.
- Useful for analytics, quotas and usage tracking.

Redis for Rate Limiting

Limit how many requests a user can make in a given time window (e.g. 100 requests per minute).

```
INCR ratelimit:user:42
EXPIRE ratelimit:user:42 60
```

- Count requests per user or IP.
- Set a time window with TTL (e.g. 60 seconds).
- Block or throttle requests when the limit is reached.

How It Works (Example Flow)

Client (Makes request) → Application (Checks session, updates counter, checks rate limit) → Redis (Sessions, Counters, Rate Limit)

Concurrency and Atomicity

- Redis operations (e.g. INCR, DECR, SET) are atomic.
- They are safe under high concurrency.
- Use atomic commands or Lua scripts for more complex logic (e.g. check and set in one step).

Common Use Cases

Use Case	Example
User Sessions	Store logged-in user info with TTL (e.g. 1 hour).
Counters	Count page views, likes or login attempts (e.g. INCR page:views:1001).
Rate Limiting	Allow max 100 requests per minute per user (e.g. INCR + EXPIRE 60).

Key Takeaways

- Redis can store sessions, counters and rate limits in memory.
- Use TTL to automatically expire data.
- Redis operations are atomic and handle high concurrency.
- These patterns help build fast, scalable and reliable applications.

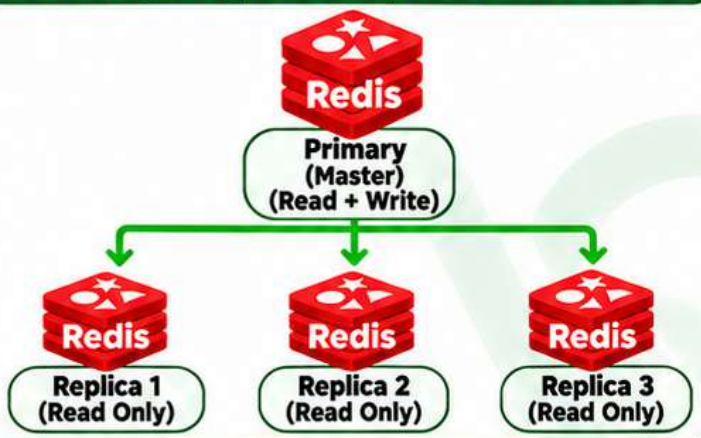
Redis Replication Explained

The Big Picture

Redis replication creates copies of your data from a primary (master) to one or more replicas, so you can scale reads and improve availability.

- Primary handles writes
- Replicas receive updates asynchronously
- Replicas serve read traffic
- Improves scalability and recovery

Primary–Replica Replication Model



How Replication Works

1. A client writes data to the primary (e.g. SET user:1 Alice).
2. The primary records the change in its replication stream.
3. Replicas pull and apply the changes asynchronously.
4. Replicas keep a copy of the data for reads, scaling and recovery.

Asynchronous Updates



- Updates are sent to replicas in the background.
- The primary does not wait for replicas to confirm.
- This keeps write performance fast and low latency.

Replication Lag



- Replication lag is the delay between a write on the primary and when it appears on a replica.
- It is usually small (milliseconds), but can increase under high load or network issues.

Why Use Replicas?

- **Read Scaling** – spread read traffic across multiple replicas.
- **Higher Availability** – keep serving reads if needed.
- **Faster Recovery** – a replica can be promoted to primary if the primary fails.

Key Takeaways

- Redis uses a primary–replica model.
- Updates are asynchronous.
- Replication lag is a normal and expected delay.
- Replicas help with read scaling and recovery.
- A replica can be promoted to primary during failover.

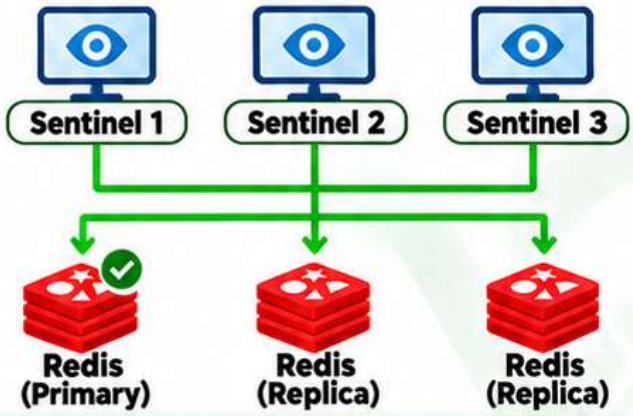
Redis Sentinel and Automatic Failover

The Big Picture

Redis Sentinel provides high availability for Redis by monitoring instances, detecting failures and automatically promoting a replica to a new primary.

- Monitors Redis instances
- Detects primary failures
- Elects a new primary
- Updates clients about the new primary

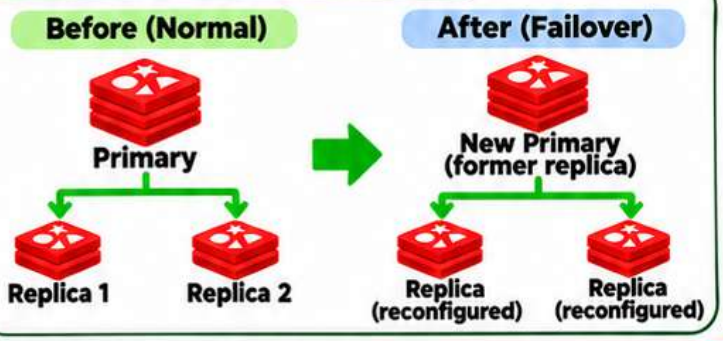
How Redis Sentinel Works



Failover Process (Step by Step)

1. Sentinels monitor the primary and replicas.
2. A failure is detected and confirmed by a quorum (e.g. 2 of 3 Sentinels).
3. A replica is elected as the new primary (automatic failover).
4. Other replicas are reconfigured to follow the new primary.
5. Clients are notified of the new primary so they can reconnect.

Cluster After Failover



What Applications Must Handle

- Reconnect when the primary changes.
- Use Sentinel-aware clients (e.g. redis-py, Jedis, go-redis).
- Be ready for brief connection disruptions during failover.
- Avoid assuming a fixed primary address.

Key Benefits

- Higher availability for Redis.
- Automatic recovery from failures.
- Minimal manual intervention.
- Works with standard Redis (replication + Sentinel).

Key Takeaways

- Redis Sentinel monitors, detects and recovers from failures.
- A replica is automatically promoted to primary.
- Clients must reconnect and use Sentinel-aware configuration.
- Sentinel gives high availability, not sharding.

Redis Cluster and Hash Slots

The Big Picture

Redis Cluster is a built-in way to scale Redis horizontally by splitting the data across multiple nodes using 16,384 hash slots.

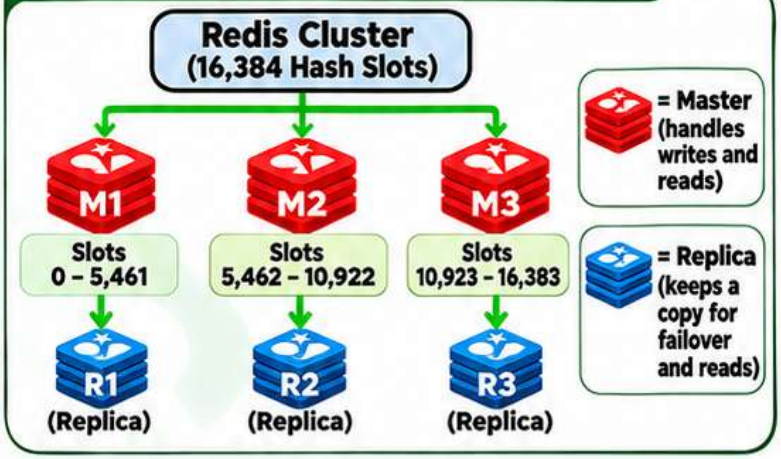
- Distributes data automatically
- Uses 16,384 hash slots
- Scales across multiple nodes
- Handles node failures
- Keeps the cluster available

Hash Slots Explained

Key "user:1001" → CRC16 Hash Function → Hash Slot (e.g. 5478)

- Redis Cluster has 16,384 hash slots (numbered 0 to 16,383).
- A key is mapped to a slot using CRC16 (key) % 16,384.
- Each slot is owned by one node in the cluster.
- The slot determines where the key is stored.

Redis Cluster Architecture



Handling Node Failures



Master Fails (e.g. M1) → Replica Becomes New Master (Automatic) → A New Replica Is Added (to restore redundancy)

- If a master node fails, one of its replicas is promoted to a master automatically.
- The affected hash slots remain available.
- A new replica can be added to maintain redundancy.

Key Benefits



- Horizontal scaling (large data and high traffic).
- High availability (automatic failover).
- Built-in data distribution (hash slots).
- No single point of failure.

Key Takeaways

- Redis Cluster splits data across 16,384 hash slots.
- Keys are mapped to slots using CRC16.
- It scales horizontally and stays available even when nodes fail.

Redis Security and Access Control

The Big Picture

Secure Redis by restricting network access, requiring authentication, using ACLs, enabling TLS, protecting secrets and applying least privilege.

- Keep Redis private and restricted
- Require authentication
- Use ACLs to control what users can do
- Encrypt traffic with TLS
- Protect secrets and use least privilege

Network Access (Keep It Private)



- Do not expose Redis to the internet.
- Allow access only from trusted networks (e.g. VPC, private subnet, security group).
- Use a firewall or cloud security group to restrict IPs and ports (e.g. 6379 or 6380).

Authentication and ACLs

```
requirepass "StrongPassword!23"
ACL: user app on +get +set +del
```

- Require a strong password (requirepass).
- Use Redis ACLs to create users with specific permissions.
- Grant only the commands needed (e.g. +get +set +del).
- Avoid using the default user in production.

TLS (Encrypt the Traffic)

TLS

- Enable TLS to encrypt data in transit.
- Use TLS for both client and replica traffic.
- Use valid certificates and rotate them before expiry.

Secret Handling

- Store Redis passwords and certificates in a secret manager (e.g. AWS Secrets Manager, Vault).
- Do not hardcode secrets in code, Dockerfiles or config files.
- Rotate credentials regularly.

Least Privilege

- Give each application only the minimum required permissions.
- Use separate ACL users for different services (e.g. app, jobs, read-only).
- Limit admin commands (e.g. CONFIG, FLUSHALL) to trusted operators only.

Key Takeaways

- Keep Redis on a private network.
- Use authentication and ACLs.
- Encrypt all traffic with TLS.
- Manage secrets securely and rotate them.
- Apply least privilege for safer operations.

Troubleshooting Redis Connection Failures

The Big Picture

Redis connection failures can happen for many reasons, such as the server not running, network blocks, wrong port, DNS issues or authentication errors.



- Identify the failure type
- Check server, network and port
- Verify DNS and connectivity
- Confirm authentication (password/ACL)
- Fix network or access restrictions

Common Redis Connection Failures

Connection Refused	Redis is not running, wrong host or port.
Timeout	Network delay, firewall, or unreachable server.
DNS Problem	Hostname cannot be resolved to an IP address.
Incorrect Port	Using the wrong port (e.g. 6379 vs 6380).
Authentication Failure	Wrong password or ACL permission denied.
Network Restriction	Firewall, security group or VPC rules blocking access.

Troubleshooting Flow

Redis Connection Fails

↓

Check if Redis is running

↓

Verify host, port and DNS

↓

Validate authentication (password/ACL)

↓

Check network rules (firewall, security group, VPC)

↓

Apply the fix and test the connection

Key Checks and Fixes

Check	What to Do
Redis Status	Confirm Redis is running (e.g. systemctl status redis).
Connectivity	Test network and port (e.g. redis-cli -h <host> -p <port> ping).
DNS	Resolve the hostname (e.g. nslookup redis.example.com).
Authentication	Use the correct password or ACL user (e.g. AUTH <password> or ACL USER).
Network Rules	Ensure firewall, security group or VPC allows the Redis port (e.g. 6379).

Useful Commands

- redis-cli -h <host> -p 6379 ping
- redis-cli -h <host> -p 6379 auth <password>
- nslookup <redis-host>
- telnet <host> 6379
- systemctl status redis
- redis-cli info (after a successful connection)

Key Takeaways

- Redis connection failures are usually caused by server, network, DNS, port, authentication or access rules.
- Follow a simple, systematic check to find and fix the root cause.
- Keep Redis accessible only from trusted networks and use authentication.

Redis Memory Problems and Performance

The Big Picture

Redis is fast, but poor memory management, wrong command patterns or high client pressure can lead to high memory usage, evictions and latency.

- Keep memory under control
- Avoid unnecessary evictions
- Find and fix slow commands
- Monitor latency and client load
- Use safe and efficient command patterns

Common Memory and Performance Problems

High Memory Usage	Redis is using too much RAM (large keys, no expiry or growing data).
Evictions	Redis removes keys because the memory limit is reached.
Slow Commands	Commands take too long (e.g. large keys or heavy operations).
High Latency	Responses are slower than expected (e.g. CPU, I/O or blocking commands).
Client Pressure	Too many connections or high request rates (overloads Redis).

Investigating the Problem (Steps)

1. Check memory usage (e.g. INFO memory).
2. Look for evictions (e.g. INFO stats | evicted_keys).
3. Find slow commands (e.g. SLOWLOG GET).
4. Check latency (e.g. redis-cli --latency).
5. Review client connections (e.g. CLIENT LIST).

Eviction and Memory Policies

- Set a memory limit (maxmemory).
- Choose an eviction policy (e.g. allkeys-lru, volatile-lru).
- Use key expiry (TTL) to remove old or unused data.

Slow and Dangerous Command Patterns

- Avoid blocking or heavy commands (e.g. KEYS *, FLUSHALL, FLUSHDB).
- Avoid large operations on big datasets (e.g. SMEMBERS, LRANGE without limits).
- Use safer alternatives (e.g. SCAN, paginated reads, smaller keys).

Performance Tuning Tips

- Keep key sizes small and efficient.
- Use appropriate data structures (e.g. HASH, SET, ZSET).
- Enable and monitor slowlog.
- Set sensible timeouts and connection limits.

Key Takeaways

- Monitor memory, evictions, latency and slow commands regularly.
- Avoid dangerous command patterns.
- Use TTLs and the right eviction policy.
- Manage client connections and load.
- Tune Redis for your workload and data size.

Monitoring Redis in Production

The Big Picture

Monitoring Redis in production helps you see its health, performance and capacity in real time, catch issues early and keep your applications reliable.



- Track Redis health and usage
- Find slow or problematic commands
- Measure latency and performance
- Use Prometheus and Grafana for visibility
- Set alerts for early issue detection

Key Redis Monitoring Tools

INFO

- Shows the overall Redis status and metrics.
- Key data: memory, clients, commands, replication, AOF, RDB and more.
- Example: INFO memory, INFO stats.

SLOWLOG

- Records commands that take longer than a configured threshold.
- Helps you find slow or inefficient operations.
- Example: SLOWLOG GET 10 to view the latest slow commands.

LATENCY

- Measures Redis internal latency for key events (e.g. command, fork, AOF).
- Helps you understand performance spikes and their causes.
- Example: LATENCY LATEST and LATENCY HISTORY.

Prometheus and Grafana Setup



Key Metrics to Watch



- Memory usage and memory limit
- Number of connected clients
- Command rate (ops per second)
- Latency (p50, p95, p99)
- Key evictions and hit/miss rate
- Replication lag (if using replicas)
- RDB/AOF backup status and duration

Useful Operational Alerts



- High memory usage (e.g. > 80% of limit).
- High latency (e.g. p95 > 50 ms).
- Increase in evictions (e.g. > 0 for a sustained period).
- High number of clients (approaching the limit).
- Replication lag too high (e.g. > 5 seconds).
- Failed RDB or AOF persistence.

Key Takeaways



- Use INFO, SLOWLOG and LATENCY to understand Redis behaviour.
- Export metrics with Prometheus and visualise them in Grafana.
- Monitor key metrics such as memory, latency, evictions and replication lag.
- Set practical alerts to detect and fix problems early.

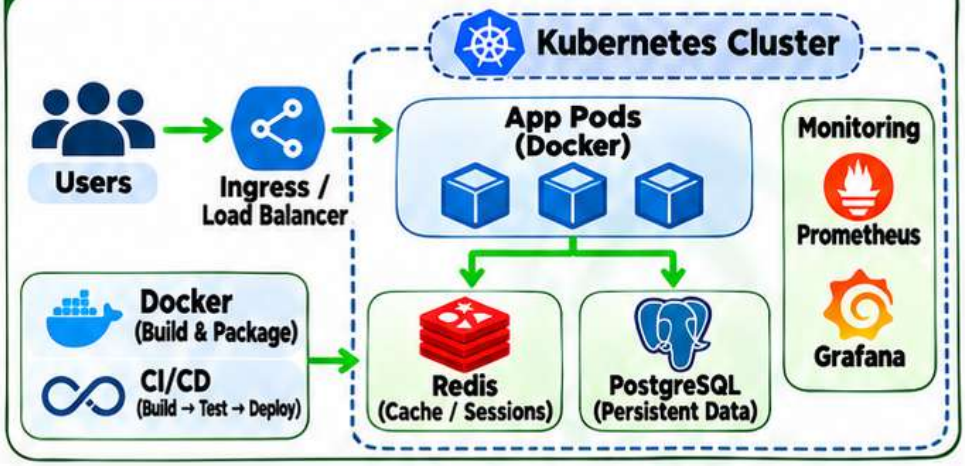
Redis in a Real DevOps Architecture

The Big Picture

Bring Redis together with an application, PostgreSQL, Docker, Kubernetes, monitoring, persistence and recovery to build a production ready, scalable and resilient system.

- Use Redis in a real production setup
- Run services with Docker and Kubernetes
- Store persistent data in PostgreSQL
- Monitor with Prometheus and Grafana
- Enable persistence, backup and recovery
- Troubleshoot a real issue like a junior engineer

End to End Architecture



Key Components

- **Redis:** cache, sessions, counters and rate limiting.
- **PostgreSQL:** primary and persistent data store.
- **Docker:** containerize the application and services.
- **Kubernetes:** run, scale and manage the services.
- **Prometheus:** collect Redis, app and cluster metrics.
- **Grafana:** dashboards and alerts for operations.

Data Flow (Simplified)

1. User sends a request to the app through the ingress.
2. App checks Redis for cached data or user session.
3. If not in cache, app reads or writes to PostgreSQL.
4. App stores frequently used data in Redis with a TTL.
5. Prometheus collects metrics and Grafana shows the system health.

Persistence and Recovery

- Enable RDB and or AOF for Redis persistence.
- Keep regular backups of Redis and PostgreSQL.
- Test recovery by restoring to a new environment.
- Use Kubernetes volume (PVC) for data durability.
- Have a clear recovery plan and RTO/RPO target.

Junior Engineering Troubleshooting Exercise

Scenario: Users report slow responses and frequent timeouts during peak traffic.

Your Task:

1. Check Redis health and memory usage.
2. Look for slow commands (SLOWLOG).
3. Verify connection limits and client load.
4. Check cache hit rate and expirations.
5. Identify the root cause and suggest a fix (e.g. increase memory, tune TTLs, optimize queries or scale the Redis pods).

Key Takeaways

- Redis fits well in a modern DevOps architecture with Docker and Kubernetes.
- Combine caching, persistent storage, monitoring and recovery for reliability.
- Use the troubleshooting steps to quickly diagnose and fix real production issues.